



Exercise 04: Eigenimages for Faces and Kaktovik Symbols

In this task, you will implement a PCA-based image recognition pipeline. You will work with two local datasets:

- face images in `eigenfaces/`
- Kaktovik symbol images in `kaktovik/`

First, you will compute a low-dimensional representation of the images using PCA (often called “eigenfaces” in the face-recognition setting). Then you will use the corresponding coefficients as features for two classifiers: `LogisticRegression` from `sklearn.linear_model` and a Gaussian Naive Bayes classifier (`GaussianNB`) from `sklearn.naive_bayes`. Finally, you will reconstruct images from a subset of principal components and compare the classification behavior on the two datasets.

Hint: This exercise builds directly on the lecture material on PCA, image representations, logistic regression, and Gaussian Naive Bayes. Please first review the relevant lecture slides and then work from the provided Python files.

Hint: You do not need to implement the evaluation metrics from scratch. You may use `sklearn.metrics`, for example `classification_report`, to obtain precision, recall, f1-score, and accuracy.

Exercise 1 Read first

The Python files already contain a lot of structure. Read the provided files first to get an overview of the pipeline, the dataset layout, and the evaluation code before you start implementing the missing parts.

Rules on imports and helper functions

In `Eigenfaces.py`, do not add additional imports and do not import functions from other local files or external libraries. Use only the modules and classes already imported in the provided skeleton. In particular:

- Allowed in `Eigenfaces.py`: NumPy operations, `np.linalg.svd`, matrix multiplication, reshaping, `LogisticRegression` from `sklearn`, and `GaussianNB` (Gaussian Naive Bayes) from `sklearn`.
- Not allowed in `Eigenfaces.py`: pre-built PCA or decomposition tools such as `sklearn.decomposition.PCA`, `TruncatedSVD`, `IncrementalPCA`, `cv2.PCACompute`, pre-built feature standardization helpers from `sklearn`, or similar shortcuts.
- Not allowed: importing helper functions from `Main.py`, the test file, or your own extra modules.
- Allowed in `Main.py`: using `sklearn.metrics` for evaluation, for example `classification_report`.

Exercise 2 Eigenfaces.py

Within the module `Eigenfaces.py`, the main tasks are:

- generating eigenimages from a set of training images,
- projecting images into PCA space,
- reconstructing an image from the first principal components,
- and training classifiers on the resulting coefficients.

The provided skeleton splits the task into helper methods and higher-level pipeline methods. Do not change the method `create_database_from_folder()`.

`calculate_average_face()`

Compute the average image over all training images. The training images are represented as row vectors. Return the average image as a one-dimensional array of length `#pixels`. Use NumPy only; do not call any pre-built dimensionality reduction method here.

`calculate_eigenfaces()`

Center the training images by subtracting the average image and compute the principal directions using SVD. Return the first requested principal components as row vectors. **Pay attention to the matrix shapes.** You may use `np.linalg.svd`, but you must not use `sklearn.decomposition.PCA` or any other pre-built PCA helper.

`get_feature_representation()`

Project every image onto the first `k` principal components and return the resulting coefficient vectors. These coefficients are the features for the classifiers. Use NumPy operations and matrix multiplication; do not import helper functions from elsewhere.

`calculate_feature_statistics()` and `standardize_features()`

Before training `LogisticRegression`, standardize the PCA coefficients feature-wise. For every feature dimension, subtract the mean value over the training set and divide by the standard deviation over the training set. After this transformation, every feature has approximately zero mean and unit variance on the training data.

Why is this useful?

- Different PCA coefficients can have very different numeric scales.
- `LogisticRegression` is usually easier to optimize when the input features have comparable scales.
- The same feature statistics computed on the training data must later be reused for every test image.

Implement this step yourself with NumPy only:

- `calculate_feature_statistics()` should compute the mean and standard deviation of every feature over the training set.

- `standardize_features()` should apply this transformation to a feature matrix.
- If a feature has standard deviation zero, replace this value by one to avoid division by zero.

Do not call a pre-built sklearn standardization helper directly in your solution.

process_and_train()

Compute the average image, the principal components, and the PCA features. Then train one classifier on top of these features. In this exercise, the supported classifier types are `logistic` and `gaussian_nb`. At this stage, you may use the `fit()` method of the provided `LogisticRegression` and `GaussianNB` (Gaussian Naive Bayes) classes from sklearn. For `logistic`, apply your own feature standardization before calling `fit()`.

train_both_classifiers()

Train both classifiers on the same PCA features:

- `LogisticRegression` from sklearn
- `GaussianNB` (Gaussian Naive Bayes) from sklearn

This lets you compare their behavior directly on the same feature space. Do not replace these classifiers by other sklearn models. Use your own standardization for `LogisticRegression`, but leave the `GaussianNB` features unchanged.

reconstruct_image()

Reconstruct a given image from its PCA coefficients using the first `k` principal components. If you use enough components, the reconstruction should become very similar to the input image. Use your own PCA feature representation and NumPy operations for the reconstruction.

classify_image()

Project a single input image into PCA space and predict its class label using one of the trained classifiers. Return the classifier output without modifying it. At this stage, using the classifier method `predict()` is intended and allowed. If `logistic` is used, apply the same feature standardization as during training before calling `predict()`.

Exercise 3 Main.py and Evaluation

Use the provided `Main.py` to evaluate your implementation on both datasets. The script loads the images from the local folder structure, performs a train/test split, trains the classifiers, reconstructs one example image, and prints classification reports. You may use the given evaluation code as it is; you do not need to implement precision, recall, f1-score, or accuracy yourself for this exercise.

Compare:

- faces vs. Kaktovik symbols
- `LogisticRegression` vs. `GaussianNB` (Gaussian Naive Bayes)

Your output should resemble the example in Fig. 1. Small differences are normal when you change parameters or the data split.

```
Example output summary

Faces      test images: 42   PCA components: 50
  LogisticRegression accuracy = 0.93
                    macro    p/r/f1 = 0.95 / 0.92 / 0.92
                    weighted p/r/f1 = 0.96 / 0.93 / 0.93
  GaussianNB      accuracy = 0.81
                    macro    p/r/f1 = 0.87 / 0.81 / 0.80
                    weighted p/r/f1 = 0.88 / 0.81 / 0.81

Kaktovik   test images: 160  PCA components: 20
  LogisticRegression accuracy = 0.90
                    macro    p/r/f1 = 0.91 / 0.90 / 0.90
                    weighted p/r/f1 = 0.91 / 0.90 / 0.90
  GaussianNB      accuracy = 0.83
                    macro    p/r/f1 = 0.84 / 0.83 / 0.83
                    weighted p/r/f1 = 0.84 / 0.83 / 0.83
```

Figure 1: Example summary scores for the current local face and Kaktovik datasets using PCA features, `LogisticRegression`, and `GaussianNB` (Gaussian Naive Bayes).

Exercise 4 Short Discussion

Write a short comment on one observation you made. For example:

- How many principal components were sufficient for a good reconstruction?
- Which classifier worked better on which dataset?
- Do the face images and the Kaktovik symbols behave similarly in PCA space?